Scheduling with superpowers

Using sched_ext to get bullet-proof latency

David Vernet
Kernel engineer

Meta

Agenda

Background and motivation

Building schedulers with sched_ext

Linux gaming under pressure

Current status and future plans

Questions?



01 Background and motivation



Preface: Did a sched_ext talk at KR in 2023

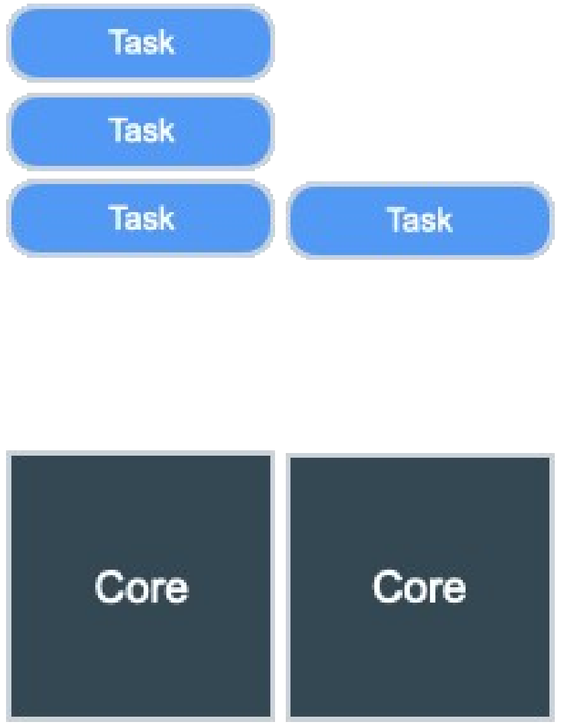
- This talk will be much more focused on general scheduling problems + real-world case studies with sched_ext
- Last year's talk: https://kernel-recipes.org/en/2023/schedule/sched_ext-pluggable-scheduling-in-the-linux-kernel/
  - Go there to learn about sched_ext interfaces and how to build a sched_ext scheduler
- This year: watch on if you want a deep dive on scheduling in general, and scheduling as it relates to Linux gaming

What is a CPU scheduler?



CPU schedulers multiplex threads onto core(s)

- Manages the finite resource of CPU between all of the execution contexts on the system
- Decide who gets to run next, where they run, and for how long
- Does context switching



Things get very complicated very quickly

- Challenging technical problem
  - **Fairness**: Everyone should get *some* CPU time
  - **Optimization**: Make optimal use of system resources, minimize critical sections
  - **Low overhead**: Should run for as short as possible
  - **Generalizable**: Should work on every architecture, for every workload, etc.



Introducing: sched_ext



sched_ext enables scheduling policies to be implemented in BPF programs

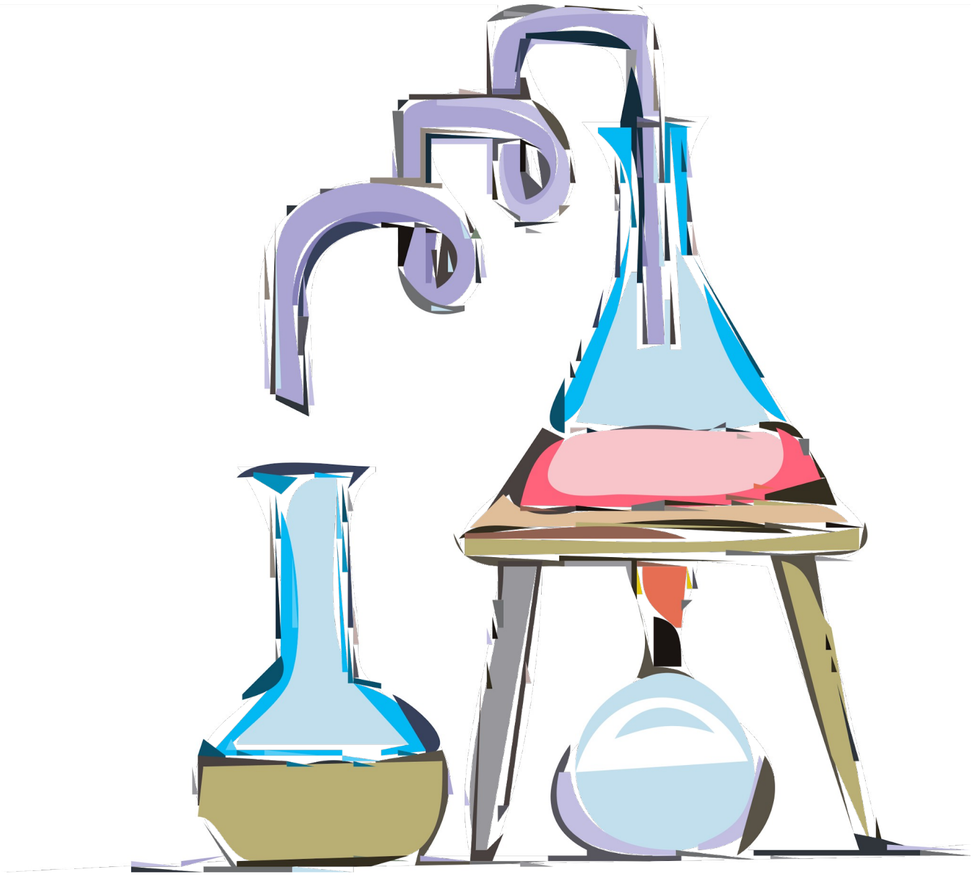
1. Write a scheduler policy in BPF
2. Compile it
3. Load it onto the system, letting BPF and core sched_ext infrastructure do all of the heavy lifting to enable it

- New sched_class, at a lower priority than CFS
- GPLv2 only



Rapid experimentation

- No reboot needed – just recompile BPF prog and reload
- Simple and intuitive API for scheduling policies
  - Does not require knowledge of core scheduler internals
- Safe, cannot crash the host
  - Protection afforded by BPF verifier
  - Watchdog boots sched_ext scheduler if a runnable task isn't scheduled within some timeout
  - New sysrq key for booting sched_ext scheduler through console
- See what works, then implement features in CFS



Bespoke scheduling policies

- CFS is a general purpose scheduler. Works OK for most applications, not optimal for many
- Linux gaming workloads with heavy background CPU pressure see huge improvements (more on this later)
- Optimizes some major Meta services (more on this later)
  - **HHVM optimized by 2.5-3+% RPS**
  - **Looking like a 3.6 - 10+% improvement for ads ranking**
- Google has seen strong results on search, VM scheduling with ghOSt



Moving complexity into user space

- Offload complicated logic such as load balancing to user space
- Avoids workarounds like custom threading implementations and other flavors of kernel bypass
- Use of floating point numbers
- BPF makes it easy to share data between the kernel and user space



02 Building schedulers with sched_ext



Again, see last year’s talk!

- https://kernel-recipes.org/en/2023/schedule/sched_ext-pluggable-scheduling-in-the-linux-kernel/
- Very briefly: sched_ext interface is a set of callbacks
  - Represent various stages in scheduling pipeline
  - Include hooks in various object lifecycles, e.g. tasks, cgroups, reweight, cpuset changes, etc



```
/* Return CPU that task should be migrated to on wakeup path. */
s32 (*select_cpu)(struct task_struct *p, s32 prev_cpu, u64 wake_flags);

/* Enqueue runnable task in the BPF scheduler. May dispatch directly to CPU. */
void (*enqueue)(struct task_struct *p, u64 enq_flags);

/* Complement to the above callback. */
void (*dequeue)(struct task_struct *p, u64 deq_flags);
...

/* Maximum time that task may be runnable before being run. Cannot exceed 30s. */
u32 timeout_ms;

/* BPF scheduler's name. Must be a valid name or the program will not load. */
char name[SCX_OPS_NAME_LEN];
```

From https://git.kernel.org/pub/scm/linux/kernel/git/tj/sched_ext.git/tree/kernel/sched/ext.c?h=for-6.12#n193

Lots of kernel-side improvements have landed since last year

- cpufreq integration
  - `void scx_bpf_cpuperf_set(s32 cpu, u32 perf);`
    - Set the relative performance of this CPU – matches schedutil interface
  - `u32 scx_bpf_cpuperf_cur(s32 cpu);`
    - See the performance level currently set for the specified CPU
  - `u32 scx_bpf_cpuperf_cap(s32 cpu);`
    - See the maximum capability for the specified CPU

- Dispatch Queue iterators
  - Iterate over tasks in a dispatch queue, selectively consume individual tasks

- Direct dispatch to remote CPUs from enqueue path
  - Previously not possible due to not being able to drop rq lock

Kernel Recipes

Lots of scheduler improvements as well

- scx_rusty
  - Much stronger interactivity support
  - Infeasible weights problem solved, with solution implemented
  - Case studies discussed later in presentation

- scx_lavd
  - Will soon ship as default scheduler on Linux Steam Deck
  - Performance mode vs. power savings mode
  - Built by Changwoo Min

- scx_rustland
  - Performant and robust user space rust library
  - Lots of hotpaths now in BPF
  - Built by Andrea Righi



03 Linux gaming under pressure

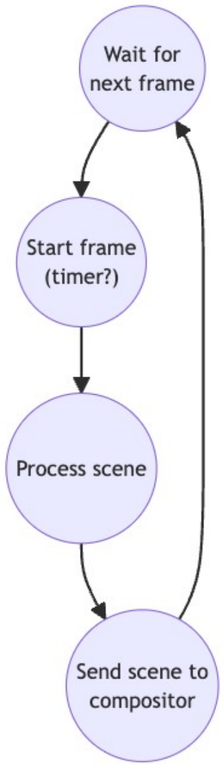
Enjoying CPU-intensive games even under horrible CPU pressure



Meta

Interactive workloads are typically *cyclic*

- Frames happen at n millisecond cadence [0]
  - Input event kicks off the rendering pipeline
    - Timer?
    - Scene input (mouse)
  - Scene is processed by application
    - Might kick off parallel work
  - Application sends completed frame to compositor
  - Application waits for new input event



[0] In games, other contexts (e.g. in VR) can be at sub-millisecond fidelity

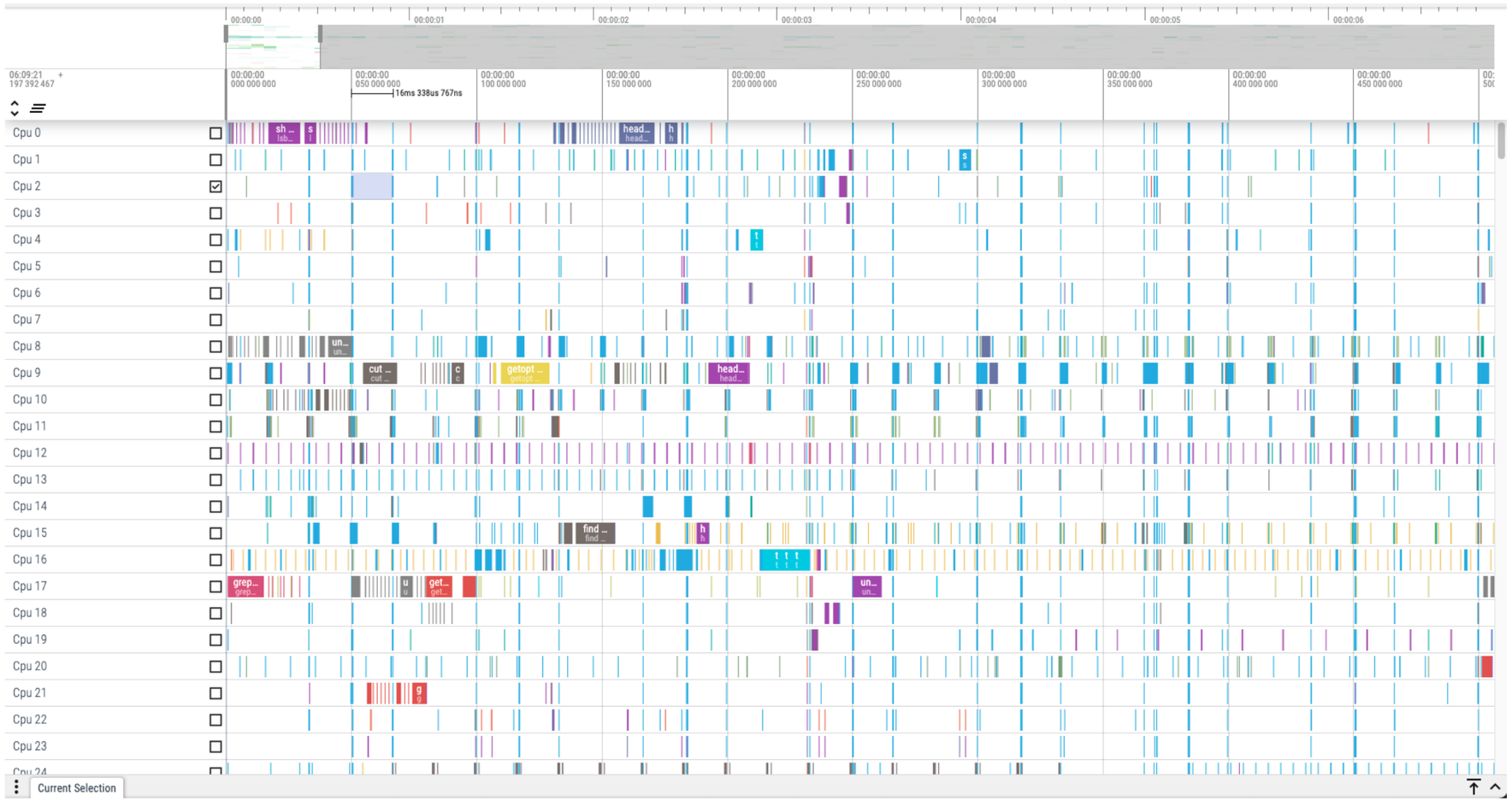
Terraria: A (sort of) simple starting example

Running with mostly idle system

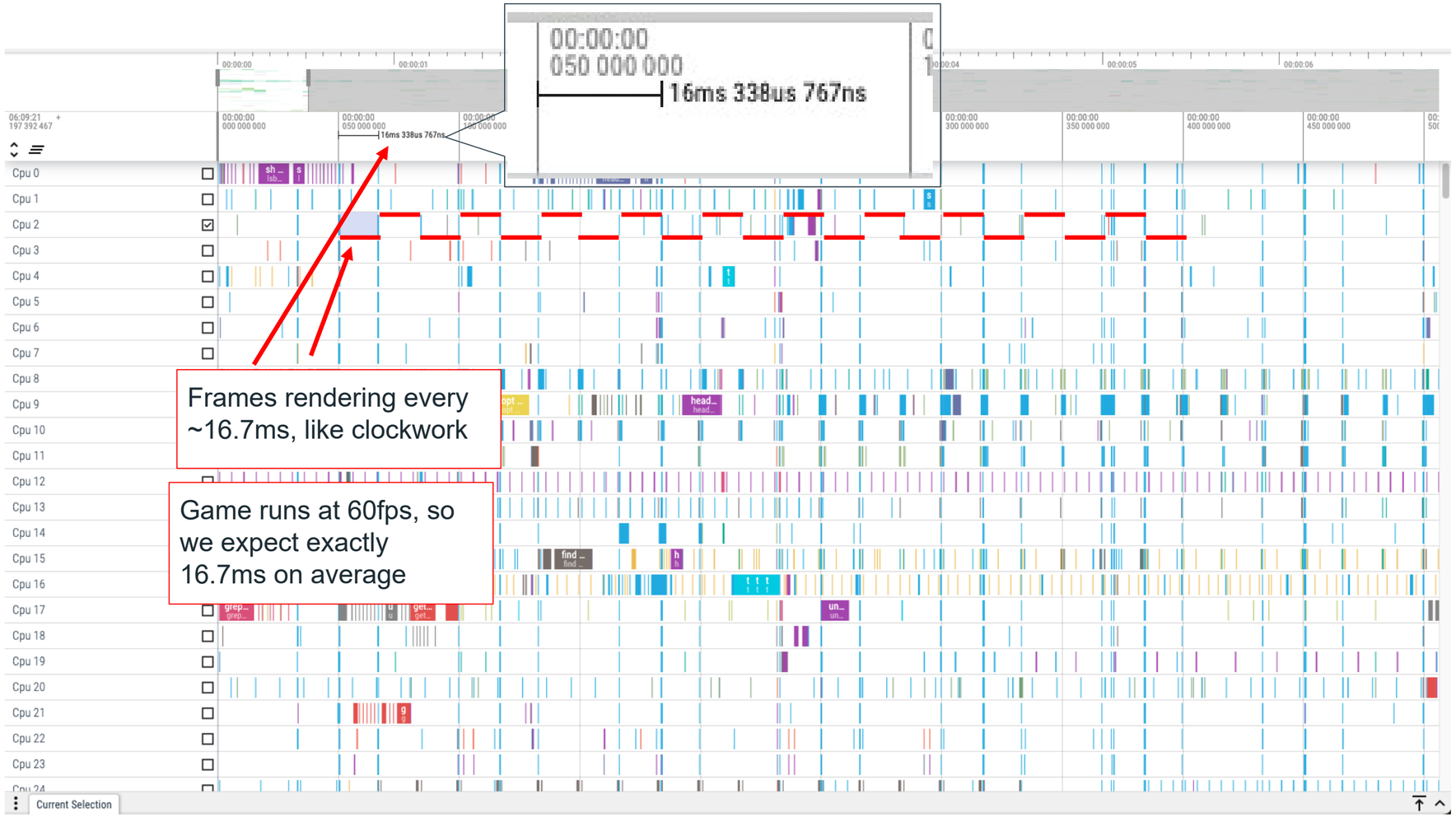
Terraria: A (sort of) simple starting example

Video link:
https://drive.google.com/file/d/1Pq0uw_T-mCqLR-g7mmDEk9Vfycmg1u8m/view?usp=sharing

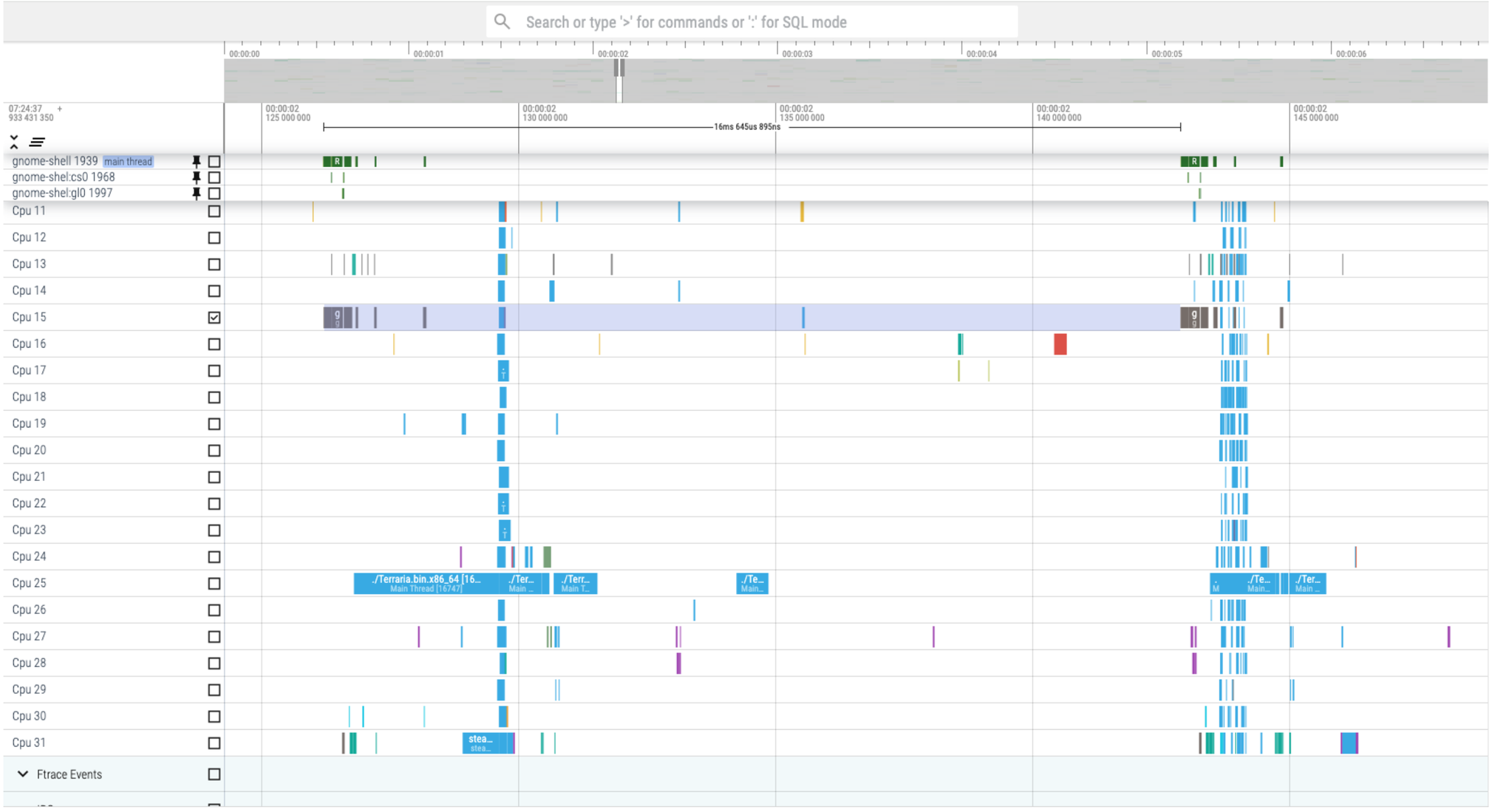
Terraria: A (sort of) simple starting example



Terraria: A (sort of) simple starting example



Terraria: A (sort of) simple starting example



Terraria: A (sort of) simple starting example

Perfetto trace demonstration

Takeaways: Highly periodic, highly pipelined

- Tasks run for typically very short bursts (often O(us))
- Lots of context switching and pipelining
- Most cores not utilized, or utilized for very short bursts
- @60 FPS, expecting roughly 16.7ms per frame
- End-to-end frame is roughly 4ms, implies max fps of ~250



What about when the system is overcommitted?

- Same deadlines exist
  - Must complete work by deadline for seamless experience
  - Beyond threshold and experience becomes unusable
- Must now contend with other threads and deal with latency issues



Terraria: A crazy, overcommit example

- Running 8x stress-ng CPU hogging workload: `stress-ng -c $((8 * $(nproc)))`
- Host: AMD Ryzen 9 7950X
  - 16 cores, 32 CPUs / hyperthreads
    - 256 CPU-hogging tasks
- Running on 6.11, using CachyOS (specifically, 6.11.0-1-cachyos-sched-ext)
  - Basically a stock 6.11 Arch Linux kernel with sched_ext enabled

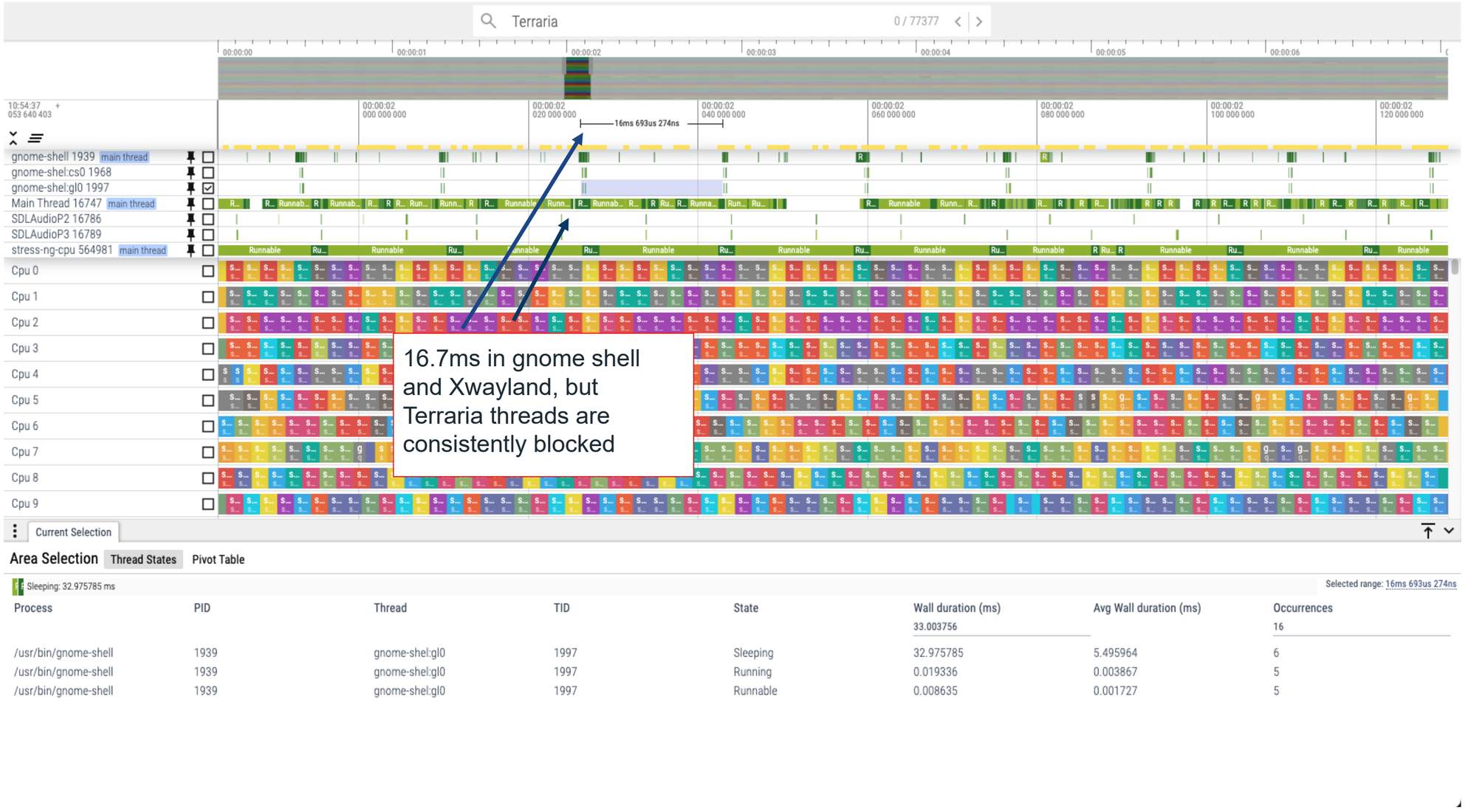
Terraria is laggy and unusable with EEVDF

Video link:
https://drive.google.com/file/d/1nYzitQVO2F2b1EibLbMvgkdaRX3SIjmZ/view?usp=sharing

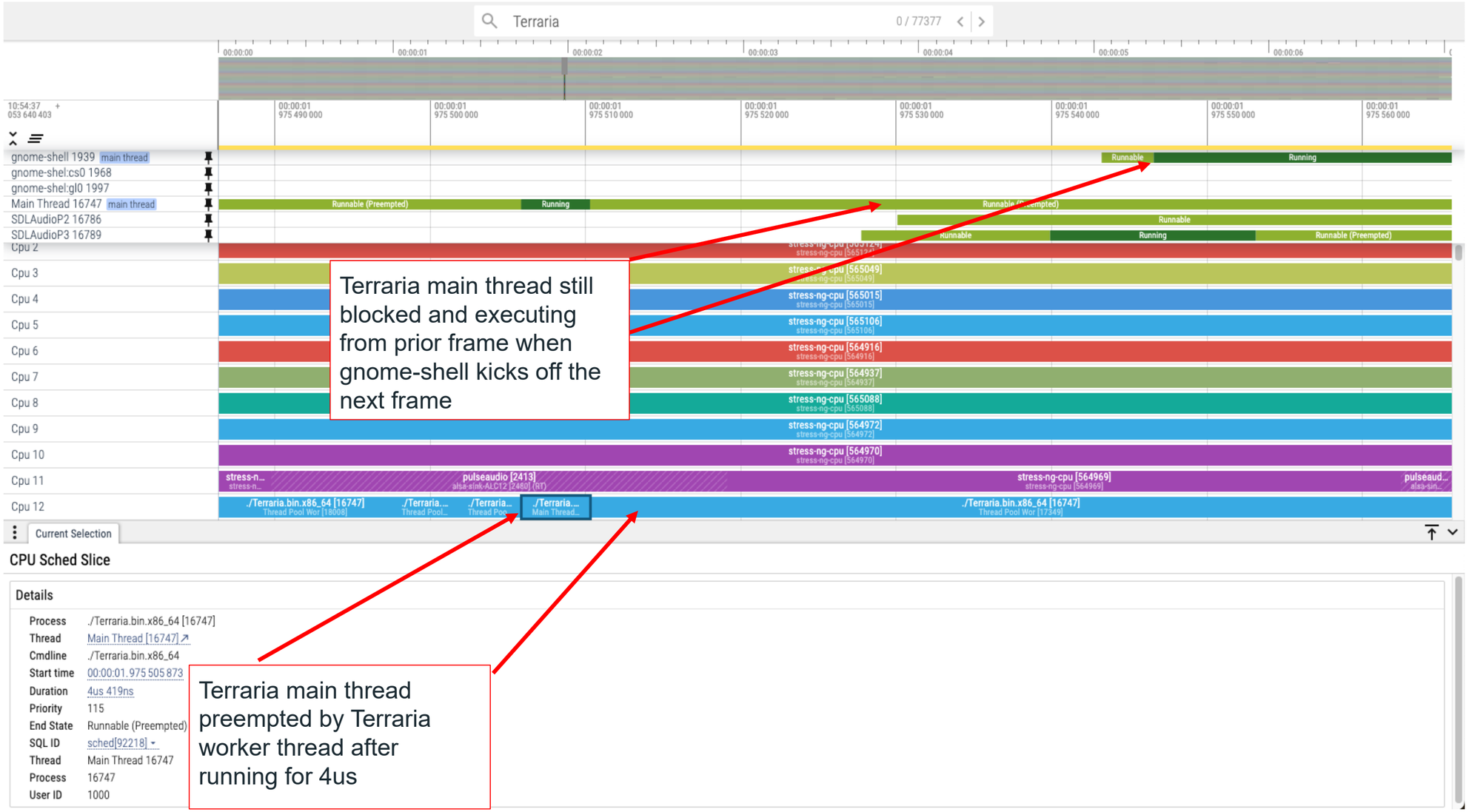
CPUs are truly stuffed



60fps still the goal, but…it’s not happening



60fps still the goal, but…it's not happening



Terraria main thread still blocked and executing from prior frame when gnome-shell kicks off the next frame

Terraria main thread preempted by Terraria worker thread after running for 4us

What’s going on here?

- stress-ng threads more hogging the entire machine
- Latency not being optimally accounted for
- To better understand, let’s look more closely at the default Linux scheduler



CFS: The **C**ompletely **F**air **S**cheduler



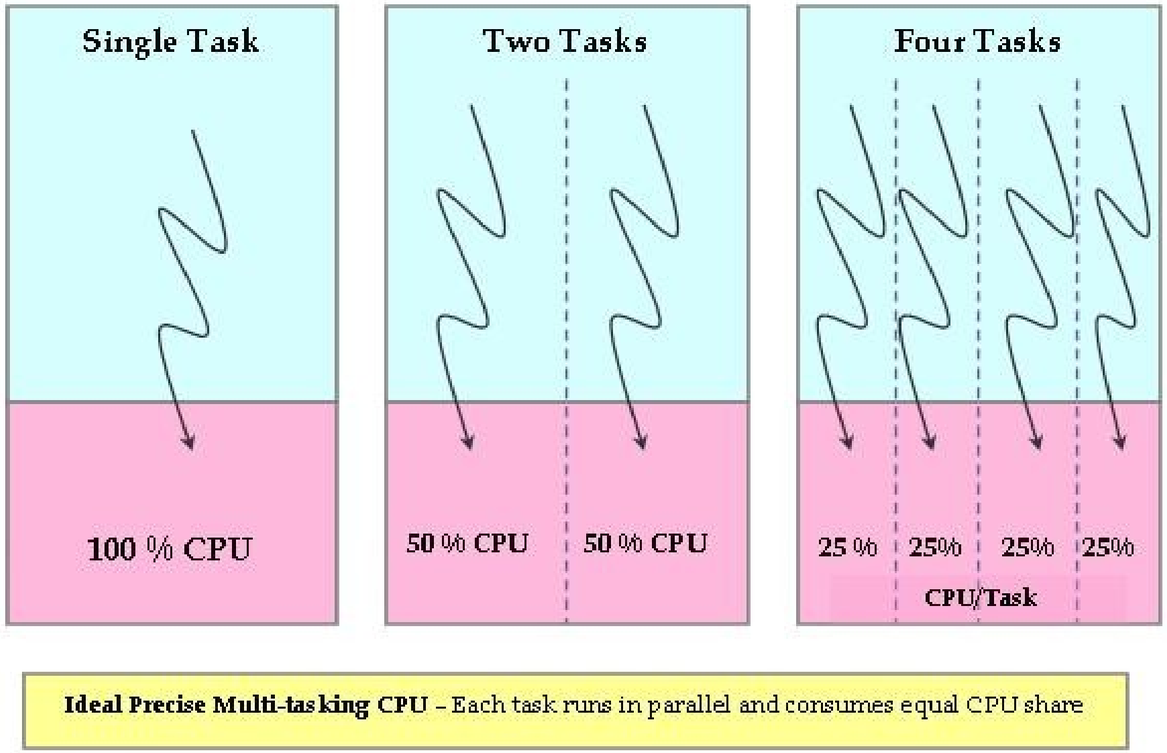
~~CFS: The **C**ompletely **F**air **S**cheduler~~

EEVDF: Earliest Eligible Virtual Deadline First



EEVDF is a “fair, weighted, virtual time scheduler”

- Threads given proportional share of CPU, according to their weight and load
  - “vruntime”
  - In example on right, all threads have equal weight
- Conceptually quite simple and elegant



Warning: math incoming

- Scheduling is inherently very mathematical
- Having at least **some** exposure to this is critical to understanding how schedulers work
- Don’t panic, not necessary to understand the math deeply. The important thing is to build an intuition.



vruntime is quite elegant: a proportional, weight-based CPU allocation

- Every task t_i has a weight w_i
- t_i ‘s allocated CPU is its proportion of weight across all tasks on the system:

$$\text{CPU time}(t_i) = w_i \cdot \int_{t_0}^{t_1} \frac{1}{\sum_{u=0}^{N} w_u}$$

vruntime is quite elegant: proportional, weight-based CPU allocation

- Every task t_i has a weight w_i
- t_i ‘s allocated CPU is its proportion of weight across all tasks on the system:

$$\text{CPU time}(t_i) = w_i \cdot \int_{t_0}^{t_1} \frac{1}{\sum_{u=0}^{N} w_u}$$

Thread t_i ‘s weight

vruntime is quite elegant: proportional, weight-based CPU allocation

- Every task t_i has a weight w_i
- t_i ‘s allocated CPU is its proportion of weight across all tasks on the system:

$$\text{CPU time}(t_i) = w_i \cdot \int_{t_0}^{t_1} \frac{1}{\sum_{u=0}^{N} w_u}$$

Inverse sum of all weights

Thread t_i ‘s weight

vruntime is quite elegant: proportional, weight-based CPU allocation

- Every task t_i has a weight w_i
- t_i ‘s allocated CPU is its proportion of weight across all tasks on the system:

$$\text{CPU time}(t_i) = w_i \cdot \int_{t_0}^{t_1} \frac{1}{\sum_{u=0}^{N} w_u}$$

Inverse sum of all weights

Thread t_i ‘s weight

Time interval [t_0, t_1)

vruntime is quite elegant: proportional, weight-based CPU allocation

- Every task t_i has a weight w_i
- t_i ‘s allocated CPU is its proportion of weight across all tasks on the system:

$$\text{CPU time}(t_i) = w_i \cdot \int_{t_0}^{t_1} \frac{1}{\sum_{u=0}^{N} w_u}$$

Inverse sum of all weights

Thread t_i ‘s weight

Time interval [t_0, t_1)

This is what fairness means in scheduling

Example: 2 tasks with weight w=1

- $w_0 = w_1 = 1$
- Both tasks get half of the CPU

$$\begin{aligned}\text{CPU time}(t_0) = \text{CPU time}(t_1) &= w_0 \cdot \int_{t_0}^{t_1} \frac{1}{\sum_{u=0}^{N} w_u} \\ &= 1 \cdot \int_{t_0}^{t_1} \frac{1}{\sum_{u=0}^{N} w_u} \\ &= \int_{t_0}^{t_1} \frac{1}{2} \\ &= \frac{t_1 - t_0}{2}\end{aligned}$$

Example: 2 tasks with weight w=1 and w=2

- $w_0 = 2$
- $w_1 = 1$

- Implication: CPU is scaled linearly by weight

$$\begin{aligned}\text{CPU time}(t_0) &= w_0 \cdot \int_{t_0}^{t_1} \frac{1}{\sum_{u=0}^{N} w_u} \\ &= 2 \cdot \int_{t_0}^{t_1} \frac{1}{3} \\ &= \frac{2}{3} \cdot (t_1 - t_0)\end{aligned}$$

$$\begin{aligned}\text{CPU time}(t_1) &= w_1 \cdot \int_{t_0}^{t_1} \frac{1}{\sum_{u=0}^{N} w_u} \\ &= 1 \cdot \int_{t_0}^{t_1} \frac{1}{3} \\ &= \frac{1}{3} \cdot (t_1 - t_0)\end{aligned}$$

Example: 2 tasks with weight w=1 and w=2

- $w_0 = 2$
- $w_1 = 1$

- Implication: CPU is scaled linearly by weight

$$\begin{aligned}\text{CPU time}(t_0) &= w_0 \cdot \int_{t_0}^{t_1} \frac{1}{\sum_{u=0}^{N} w_u} \\ &= 2 \cdot \int_{t_0}^{t_1} \frac{1}{3} \\ &= \frac{2}{3} \cdot (t_1 - t_0)\end{aligned}$$

Task 0 gets 2/3 of CPU

$$\begin{aligned}\text{CPU time}(t_1) &= w_1 \cdot \int_{t_0}^{t_1} \frac{1}{\sum_{u=0}^{N} w_u} \\ &= 1 \cdot \int_{t_0}^{t_1} \frac{1}{3} \\ &= \frac{1}{3} \cdot (t_1 - t_0)\end{aligned}$$

Task 1 gets 1/3 of CPU

How this is implemented: **vruntime**

- Add how much CPU each task has used, scale by weight
- Accumulating this way ends up being equivalent to fairness equation

- Task has run for X nanoseconds, accumulate vruntime as:

$$\mathbf{vruntime}(t_i) += \frac{\mathbf{runtime} * 100}{w_i}$$

How this is implemented: **vruntime**



- Add how much CPU each task has used, scale by weight
- Accumulating this way ends up being equivalent to fairness equation

- Task has run for X nanoseconds, accumulate vruntime as:

$$\mathbf{vruntime}(t_i) += \frac{\mathbf{runtime} * 100}{w_i}$$

NOTE: Default weight in Linux is 100

Bottom line: **vruntime** is a **weighted portioning** of CPU

- Integrals represent allocation in a perfectly fair, zero-overhead system
  - Often referred to as a “fluid flow” representation
- **Intuition**: We split the available CPU fairly amongst tasks, based on their weight
- Task with the lowest vruntime is always chosen to run on the CPU



Introducing: fair deadline schedulers

- Entity chosen based on deadline *derived* from vruntime, rather than vruntime itself



As of Linux 6.7, default scheduler is EEVDF

- **E**arliest **E**ligible **V**irtual **D**eadline **F**irst
- Still vruntime based, but schedule based on deadline *derived* from vruntime

- **E**arliest: Earliest determined deadline is chosen
- **E**ligible: Only tasks that have not received more CPU than they “should have” can run
- **V**irtual: Deadline is *virtual*. That is, it’s derived from the task’s proportion of allocated CPU, **not** from any actual wall-clock deadline
- **D**eadline: The deadline derived from vruntime
- **F**irst: Choose the earliest deadline first



Warning: more math incoming

- Same rules applies: **the important thing is to build an intuition**



EEVDF: Task's "eligible" time + its slice length (inversely weighted)

- Eligible = it hasn't received *more* CPU than it should have received up until this point
  - The scheduler might give more CPU than it should have, this is called *lag*
- A task's deadline is therefore calculated as:

$$V_x(d) = V_x(e) + \frac{r_x}{w_x}$$

Kernel Recipes

EEVDF: Task's "eligible" time + its slice length (inversely weighted)

- Eligible = it hasn't received *more* CPU than it should have received up until this point
  - The scheduler might give more CPU than it should have, this is called *lag*
- A task's deadline is therefore calculated as:

$$V_x(d) = V_x(e) + \frac{r_x}{w_x}$$

Thread's deadline



EEVDF: Task's "eligible" time + its slice length (inversely weighted)

- Eligible = it hasn't received *more* CPU than it should have received up until this point
  - The scheduler might give more CPU than it should have, this is called *lag*
- A task's deadline is therefore calculated as:

$$V_x(d) = V_x(e) + \frac{r_x}{w_x}$$

Thread's deadline

Eligible time (i.e. no lag)



EEVDF: Task's "eligible" time + its slice length (inversely weighted)

- Eligible = it hasn't received *more* CPU than it should have received up until this point
  - The scheduler might give more CPU than it should have, this is called *lag*
- A task's deadline is therefore calculated as:

$$V_x(d) = V_x(e) + \frac{r_x}{w_x}$$

"Request" length (i.e. slice length)

Eligible time (i.e. no lag)

Thread's deadline



EEVDF: Task's "eligible" time + its slice length (inversely weighted)

- Eligible = it hasn't received *more* CPU than it should have received up until this point
  - The scheduler might give more CPU than it should have, this is called *lag*
- A task's deadline is therefore calculated as:

$$V_x(d) = V_x(e) + \frac{r_x}{w_x}$$

"Request" length (i.e. slice length)

Weight (same as vruntime weight)

Eligible time (i.e. no lag)

Thread's deadline



Bottom line: wait until task is eligible to run, then use its slice length to determine deadline

- Tasks with shorter slice lengths will be scheduled earlier, but preempted more often
- Tasks with high weight (low niceness) will have an earlier deadline due to being inversely scaled



EEVDF is OK, but has shortcomings

- Deadline based on slice length is questionable
  - How do you choose a slice length? Difficult (impossible?) to reason about and tune correctly.
- Eligibility is confusing, unclear when and why it’s necessary
  - Not fully explained in EEVDF paper, but it’s not necessary for fairness
  - My guess: it’s for avoiding short slices from hogging CPU
    - Tasks with really late deadlines (long slice lengths) might have to wait a really long time to be scheduled
    - Eligibility slices up the CPU a bit more fairly
    - In practice, seems to hurt interactivity (short slices imply low latency)



Solution: Build a better deadline-based scheduler

- Deadline based on task runtime
  - Runtime tracked *by the scheduler*, no user input required (other than for weight)
- Boost threads that are part of work chains



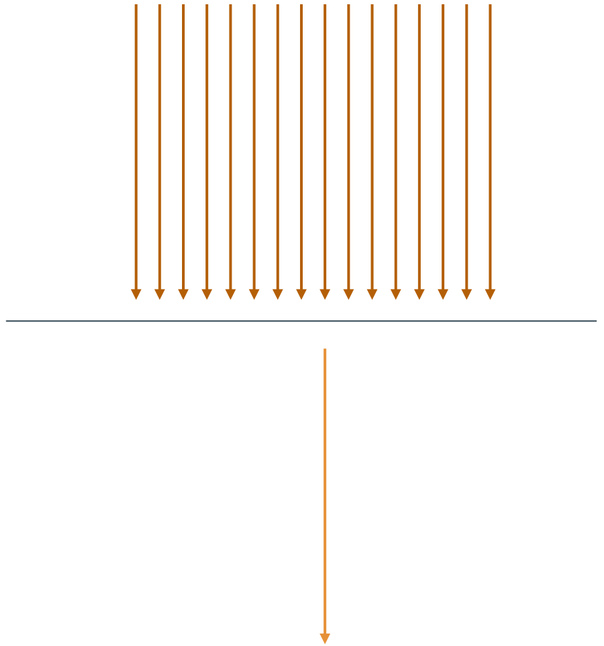
Have scheduler automatically determine deadline from task runtime statistics

- First discovered + applied by Changwoo Min @ Igalia with scx_lavd
- Tasks with high *waking* frequency: **producer task**
- Tasks with high *blocking* frequency: **consumer task**
- Tasks with high frequencies of both: **middle of a work chain**
- **Idea**: Boost (or throttle) latency priority of tasks based on these frequencies
- First added to scx_lavd scheduler, now used to run Steam Deck
  - Concepts applied to other schedulers, e.g. scx_rusty

Amdahl's Law: Serialization has high cost

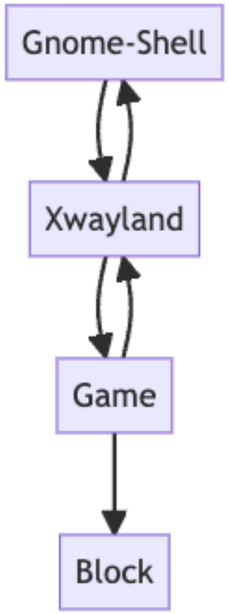
- Imagine a periodic workload that’s 50% serial, 50% highly parallel
- Optimizing either portion by the same amount will result in the *same speedup*
- Speedup of either portion given by Amdahl’s Law:

$$
\begin{aligned}
S_{lat}(x) &= \frac{1}{(1 - .5) + \frac{.5}{2}} \\
&= \frac{1}{.5 + .25} \\
&= \frac{1}{.75} \\
&\approx 33\%
\end{aligned}
$$



It’s critical that we speed up work chains

- Audio and rendering workloads are highly pipelined
- Short burst of tasks that are all on the critical path

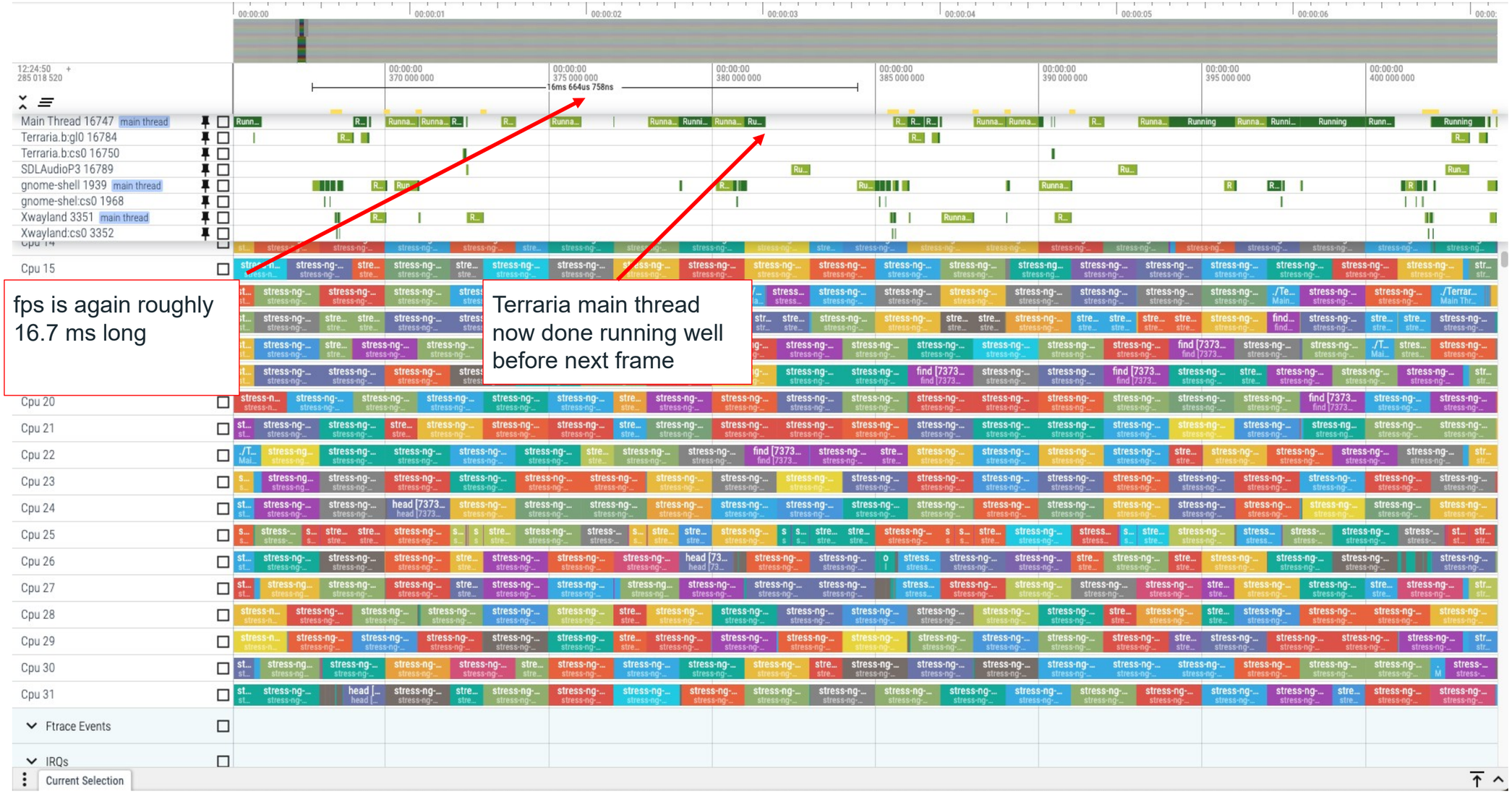


Kernel Recipes

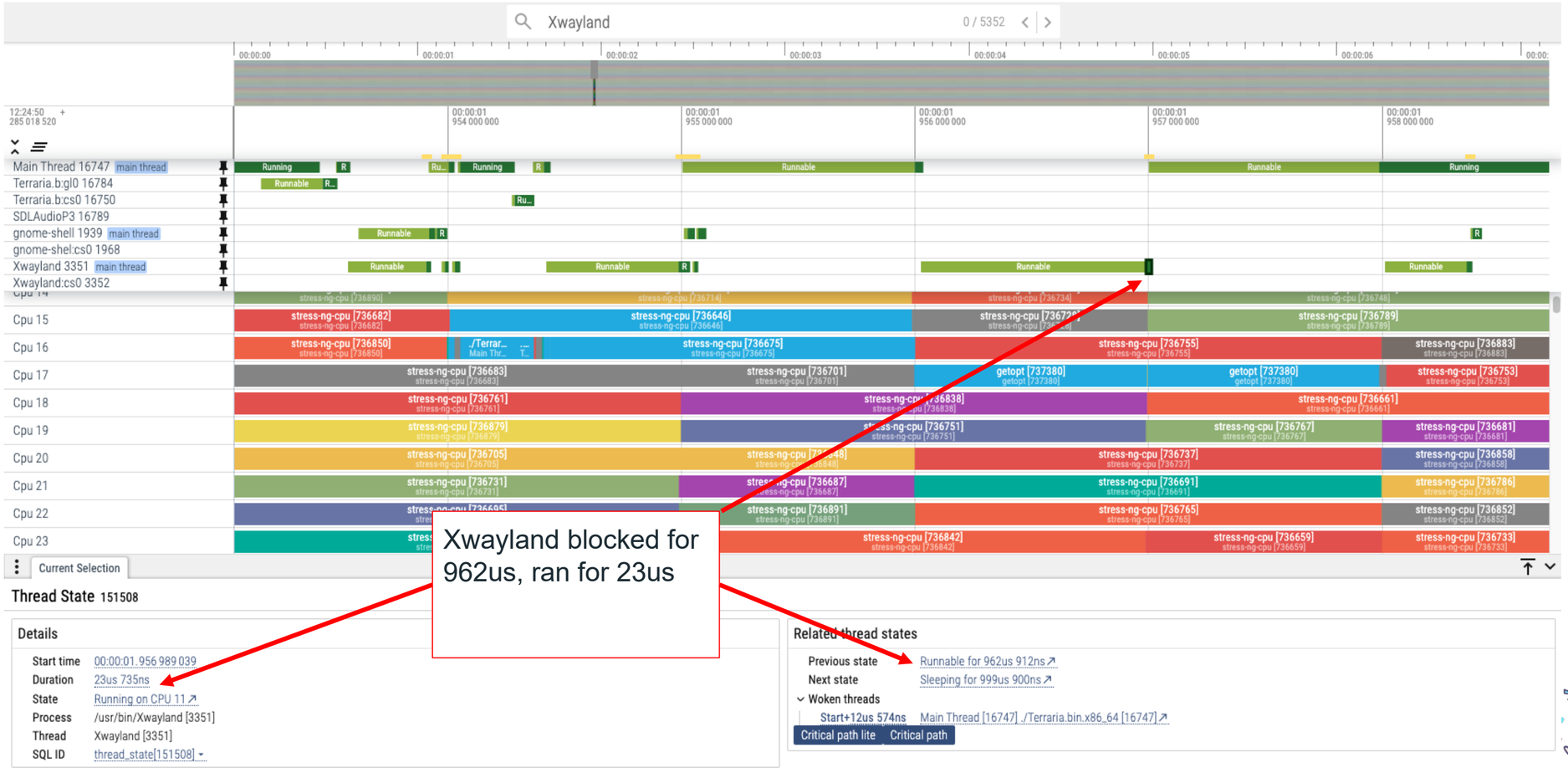
scx_rusty much more robust to background work

Video link:
https://drive.google.com/file/d/1upHlOCFyFyykVUDU3x4kgG3KZgAUwH_p/view?usp=sharing

60fps achieved, looks more like idle case



Still not perfect, note long wait times for comparatively fast runtimes





How else can we improve?

Idea 1: Deadline boost priority inheritance

- Inherit boost when a high priority / low latency task wakes up a lower-priority (longer running, etc) task
  - Hypothesis: Necessary to account for all scenarios
    - E.g.: A game with one or more CPU-hogging tasks that run for nearly the entire frame cycle
    - Baked in assumption: Priority should always be inherited
      - What about high-priority task scheduling low-priority work?

Idea 2: Cooperative scheduling

- Have user space runtime framework (e.g. folly) communicate priority to scheduler for work items
  - Executors have QoS, communicate that via BPF maps to kernel
    - Priority need not be inferred, less error prone, but requires user-space intervention
  - Will be easier to implement if and when we have hierarchical scheduling

Idea 3: Grouping work chains in cgroups

- What if we aggregate work chains of tasks that are in the same cgroup, and allow users to classify them?
  - High QoS cgroup implies more aggressive deadline boosting
- Likely works better systems with lots of periodic deadlines
- Advantage: allow system to disambiguate between multiple work pipelines
  - Common for VR workloads: lots of data pipelines with very high fidelity requirements, immersive + 2D panel applications, etc



04 Current status and future plans



Upstream status: PR ~~sent~~ **accepted** for v6.12

- PR ~~sent to~~ merged by Linus for v6.12: https://lore.kernel.org/lkml/ZuOULxIZmbvY1skN@slm.duckdns.org/
- ~~Let's keep our fingers crossed!~~
- Schedulers repo is very active: https://github.com/sched-ext/scx

lore.kernel.org/all/172693882678.2537618.10044551650165201426.pr-tracker-bot@kernel.org/

Facebook Managed | Outlook | Linux Kernel | Scheduler Tracing... | Releases · sched-... | #linux-cachyos | Workplace | Perfetto UI | RL | ODS | scx | ACM | bpftrace/reference... | sched_ext - Slack | errnos

All of lore.kernel.org

search help / color / mirror / Atom feed

From: pr-tracker-bot@kernel.org
To: Tejun Heo <tj@kernel.org>
Cc: Linus Torvalds <torvalds@linux-foundation.org>,
linux-kernel@vger.kernel.org, David Vernet <void@manifault.com>,
Ingo Molnar <mingo@redhat.com>,
Peter Zijlstra <peterz@infradead.org>,
Alexei Starovoitov <ast@kernel.org>
Subject: Re: [GIT PULL] sched_ext: Initial pull request for v6.12
Date: Sat, 21 Sep 2024 17:13:46 +0000 [thread overview]
Message-ID: <172693882678.2537618.10044551650165201426.pr-tracker-bot@kernel.org> (raw)
In-Reply-To: <ZuOULxlZmbvY1skN@slm.duckdns.org>

The pull request you sent on Thu, 12 Sep 2024 15:23:59 -1000:

> git://git.kernel.org/pub/scm/linux/kernel/git/tj/sched_ext.git/ tags/sched_ext-for-6.12

has been merged into torvalds/linux.git:
https://git.kernel.org/torvalds/c/88264981f2082248e892a706b2c5004650faac54

Thank you!

--
Deet-doot-dot, I am a bot.
https://korg.docs.kernel.org/prtracker.html

---

prev parent reply other threads:[~2024-09-21 17:13 UTC|newest]

Thread overview: 2+ messages / expand[flat|nested] mbox.gz Atom feed top
2024-09-13 1:23 [GIT PULL] sched_ext: Initial pull request for v6.12 Tejun Heo
2024-09-21 17:13 ` pr-tracker-bot [this message]

Reply instructions:

You may reply publicly to this message via plain-text email
using any one of the following methods:

* Save the following mbox file, import it into your mail client,
and reply-to-all from there: mbox

Avoid top-posting and favor interleaved quoting:
https://en.wikipedia.org/wiki/Posting_style#Interleaved_style

* Reply using the **--to**, **--cc**, and **--in-reply-to**
switches of git-send-email(1):

Upcoming features

- Hierarchical cgroup scheduling:
  - Can we build a hierarchical scheduling model where we attach schedulers to cgroups?
    - Enables building in-process schedulers, cooperating between user-space runtime and scheduler
  - Scheduler in parent cgroup chooses child, if child has a scheduler call into that cgroup’s scheduler, etc
  - Still in early design phase, no code yet
- New schedulers in the works, ideas always flowing

Links

- **Kernel** repo: https://git.kernel.org/pub/scm/linux/kernel/git/tj/sched_ext.git
- **Schedulers** repo: https://github.com/sched-ext/scx
- Documentation: https://git.kernel.org/pub/scm/linux/kernel/git/tj/sched_ext.git/tree/Documentation/scheduler/sched-ext.rst?h=for-6.12
- v.12 PR patch set: https://lore.kernel.org/lkml/ZuOULxlZmbvY1skN@slm.duckdns.org/
- Slack channel: https://bit.ly/scx_slack

Questions?